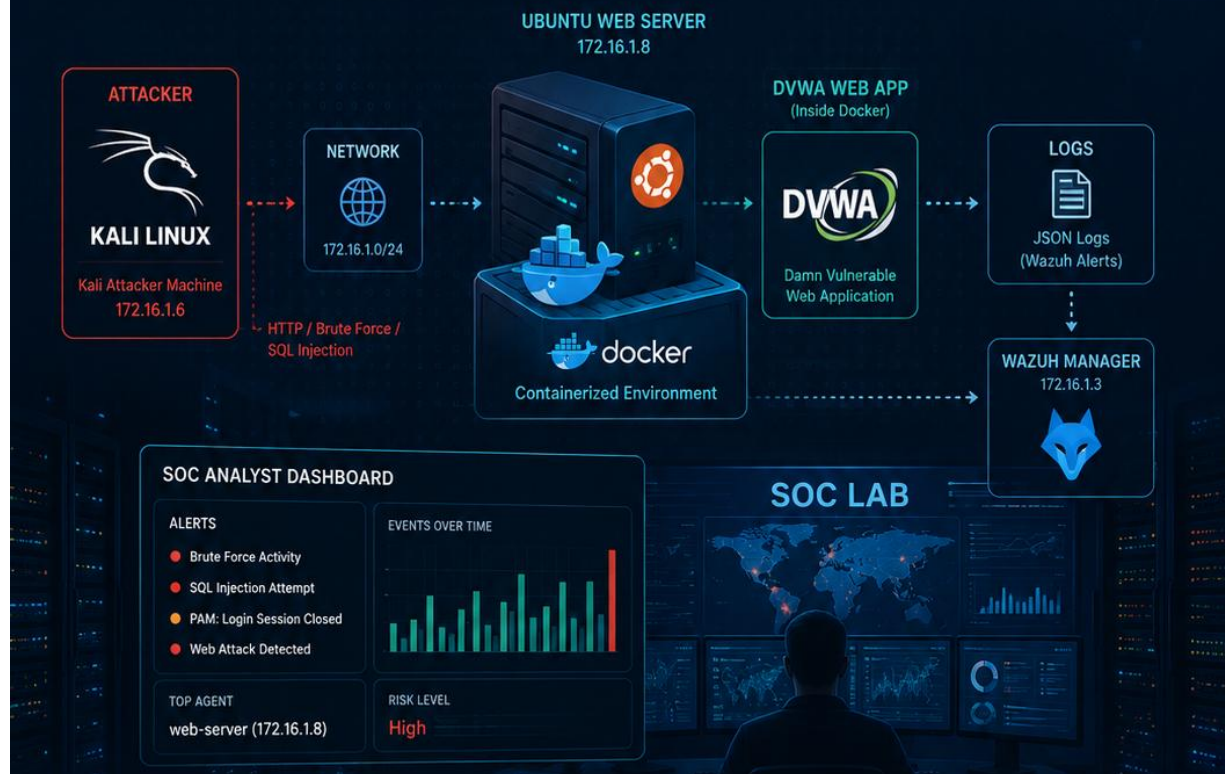


Integrating an Ubuntu Web Server into a SOC Lab: Docker and DVWA Deployment

✓ SOC Lab Project – Part 1



Integrating an Ubuntu Web Server into a SOC Lab: Docker and DVWA Deployment

RITIK PATEL

SOC – Threat Research Intern

</Hacker4Help>

Ritik Patel

SOC - Threat Research intern at </Hacker4Help>

INDEX

Sr. No	Title	Pg NO
1	Introduction	3
2	Lab Architecture	5
3	Wazuh Agent Installation	7
4	Docker Installation	15
5	DVWA Deployment	20
6	Conclusion	28

1. Introduction

In the previous phases of this SOC lab project, an enterprise-style environment was built using Active Directory, Windows and Ubuntu clients, Kali Linux, and Wazuh SIEM. After completing endpoint-based threat detection on Windows and Linux systems, this phase focuses on expanding the SOC lab by integrating a dedicated Ubuntu web server.

Web servers are an important part of real-world enterprise environments because they host websites, internal applications, databases, APIs, and other services. These systems are common targets for attackers because web applications may contain vulnerabilities such as weak authentication, SQL injection, cross-site scripting, command injection, and insecure file upload functionality.

In this part of the project, a new Ubuntu Server was installed in VirtualBox and connected to the existing SOC lab network. Docker was installed on the Ubuntu Server to provide a flexible and isolated environment for hosting web applications. DVWA was then deployed using Docker as a vulnerable web application for future attack simulation and detection activities.

The purpose of this report is to document the setup phase of the web application security lab. This includes Ubuntu Server installation, network verification, Docker installation, DVWA deployment, and successful access to DVWA from Kali Linux. The detection and analysis phase using Wazuh SIEM will be covered in the next report.

The objectives of this phase are:

- Integrate a dedicated Ubuntu web server into the existing SOC lab environment
- Verify network connectivity between Kali Linux and the Ubuntu web server
- Install Docker and Docker Compose on the Ubuntu Server
- Deploy DVWA as a containerized vulnerable web application

</Hacker4Help>

- Configure DVWA so it is accessible from Kali Linux
- Prepare the environment for future web attack simulation and Wazuh detection

2. Lab Environment

2.1 Lab Architecture

The lab environment used in this project is an extension of the previously established SOC lab setup. The existing lab already included Active Directory, endpoint devices, Kali Linux, and Wazuh SIEM. For this phase, a new Ubuntu Server was added to the same VirtualBox NAT network and used as a dedicated web server.

The Ubuntu Server was used to host DVWA using Docker. Kali Linux was used to verify access to the web application and will later be used as the attacker machine for web application attack simulation. Wazuh SIEM is already part of the overall SOC environment and will be used in the next phase for log collection and detection.

Components of the Lab:

1. Domain Controller: Windows Server with Active Directory Domain Services
2. Endpoint Machines: Windows 10 and Ubuntu client systems used in previous SOC lab phases
3. Attacker Machine: Kali Linux used to access and later attack DVWA
4. Web Server: Ubuntu Server used to host DVWA using Docker
5. Vulnerable Web Application: DVWA deployed for future web attack simulation
6. SIEM Platform: Wazuh SIEM already available for future log collection and detection
7. Network Configuration: All machines were connected within the same VirtualBox NAT network

2.2 Environment purpose

The purpose of this environment was to add a realistic web server component to the SOC lab. In real organizations, SOC analysts do not monitor only endpoint devices; they also monitor servers, web applications, authentication activity, container logs, and network-based events.

By adding an Ubuntu web server, the lab becomes more complete and closer to a real-world environment. Docker was used so that DVWA and future web applications can be deployed in isolated containers without affecting the main Ubuntu host or other applications.

2.3 Scope of the Environment

This part focuses only on the setup and deployment phase. The scope includes:

- Creating and installing the Ubuntu Server VM
- Connecting the Ubuntu Server to the existing SOC lab network
- Verifying connectivity from Kali Linux
- Installing Docker and Docker Compose
- Deploying DVWA using Docker
- Fixing DVWA port binding so it is accessible from Kali Linux
- Initializing DVWA and preparing it for future attack simulation

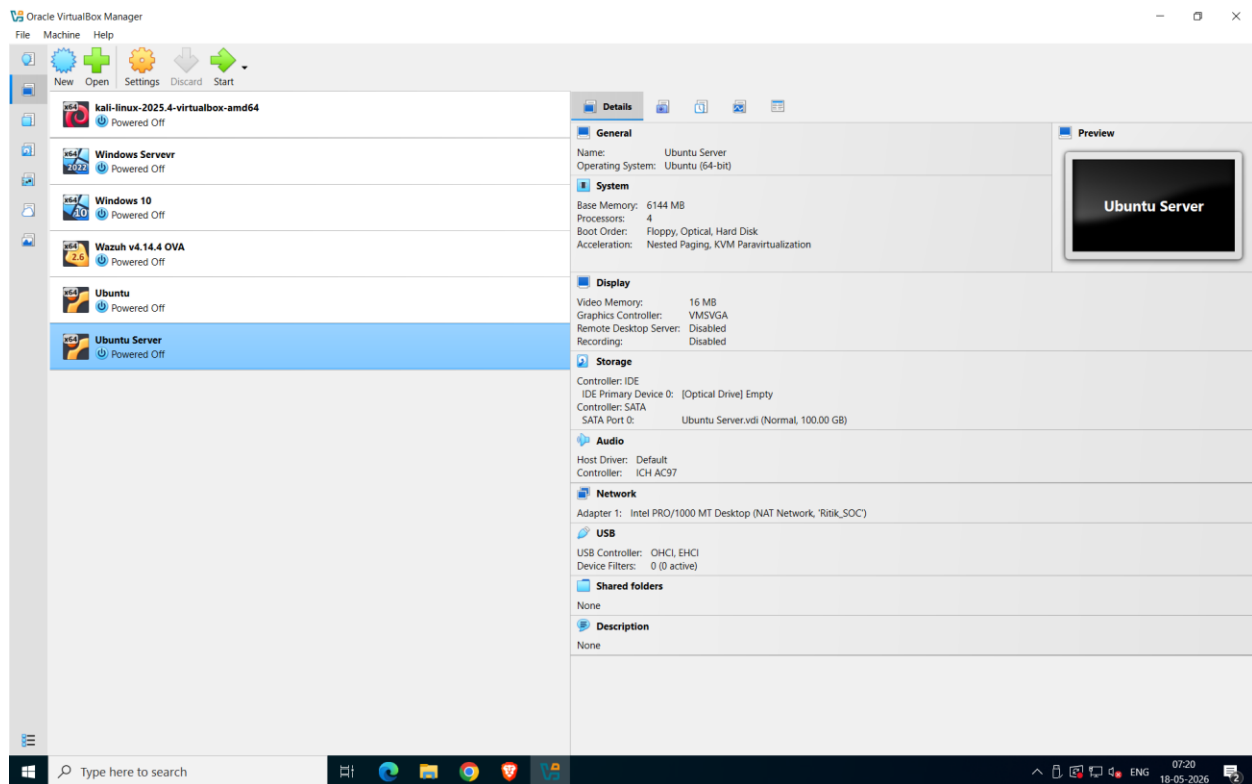
The attack simulation, Docker log collection, Wazuh local rules, and detection analysis will be covered in the next report.

3. Ubuntu Web Server Installation

3.1 Overview

For this phase of the SOC lab, a new Ubuntu Server virtual machine was installed in VirtualBox and added to the existing SOC lab network. This server was used as a dedicated web server for hosting DVWA and later collecting web application logs through Wazuh.

The reason for using a separate Ubuntu Server was to create a more realistic SOC environment where a web server can be monitored along with existing systems such as Active Directory, endpoint devices, Kali Linux, and Wazuh SIEM.



3.2 Ubuntu Server VM Configuration

A new virtual machine was created in Oracle VirtualBox using the Ubuntu Server ISO file.

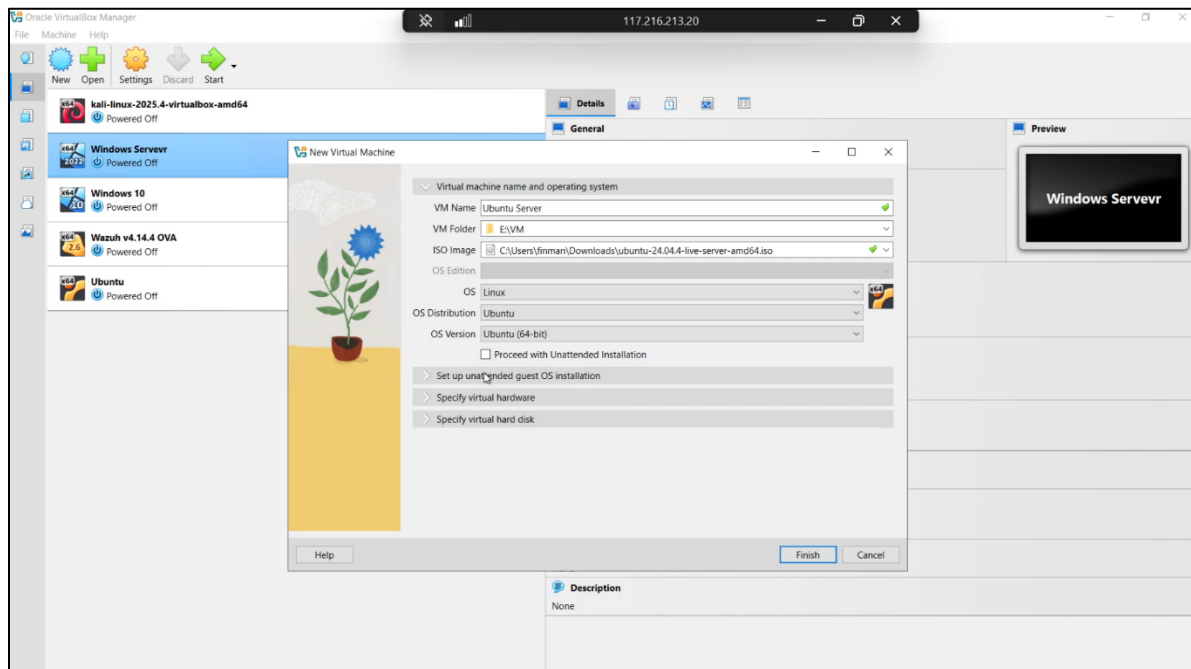
Ritik Patel

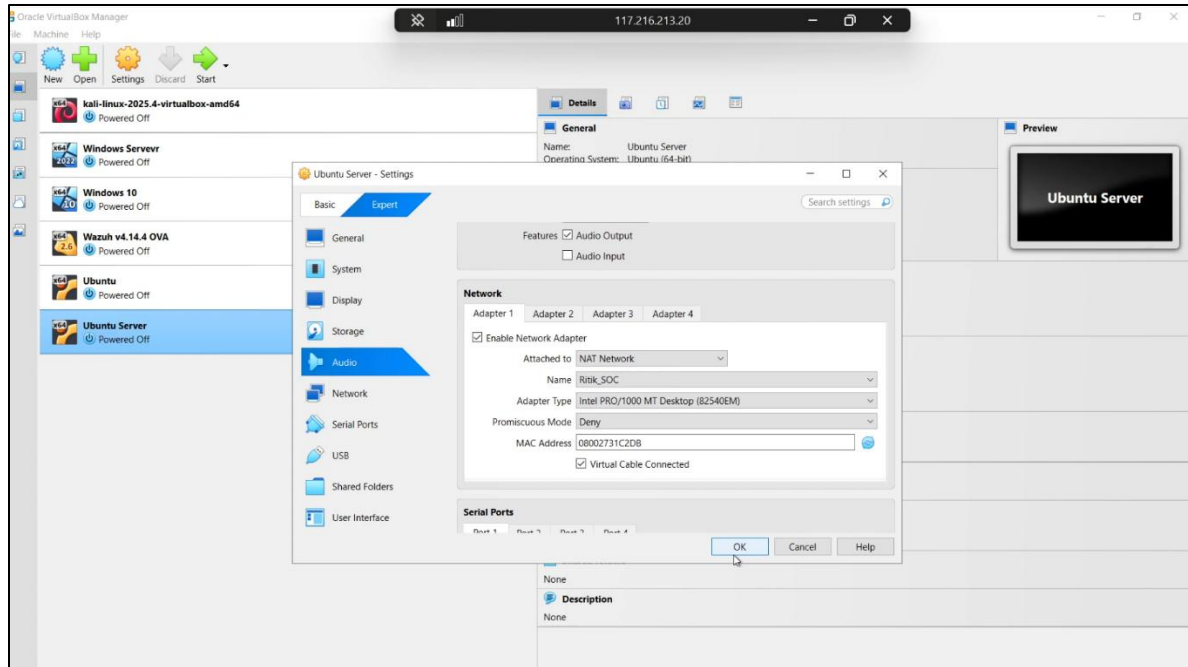
SOC - Threat Research intern at </Hacker4Help>

Virtual Machine Details:

Setting	Value
VM Name	Ubuntu Server
Operating System	Linux
Version	Ubuntu 64-bit
ISO Image	Ubuntu Server 24.04 LTS
Network Mode	Existing Ritik SOC NAT Network
Purpose	Web server for DVWA deployment

The virtual machine was connected to the same NAT network as the existing SOC lab machines so that Kali Linux, Wazuh, and the Ubuntu Server could communicate with each other.



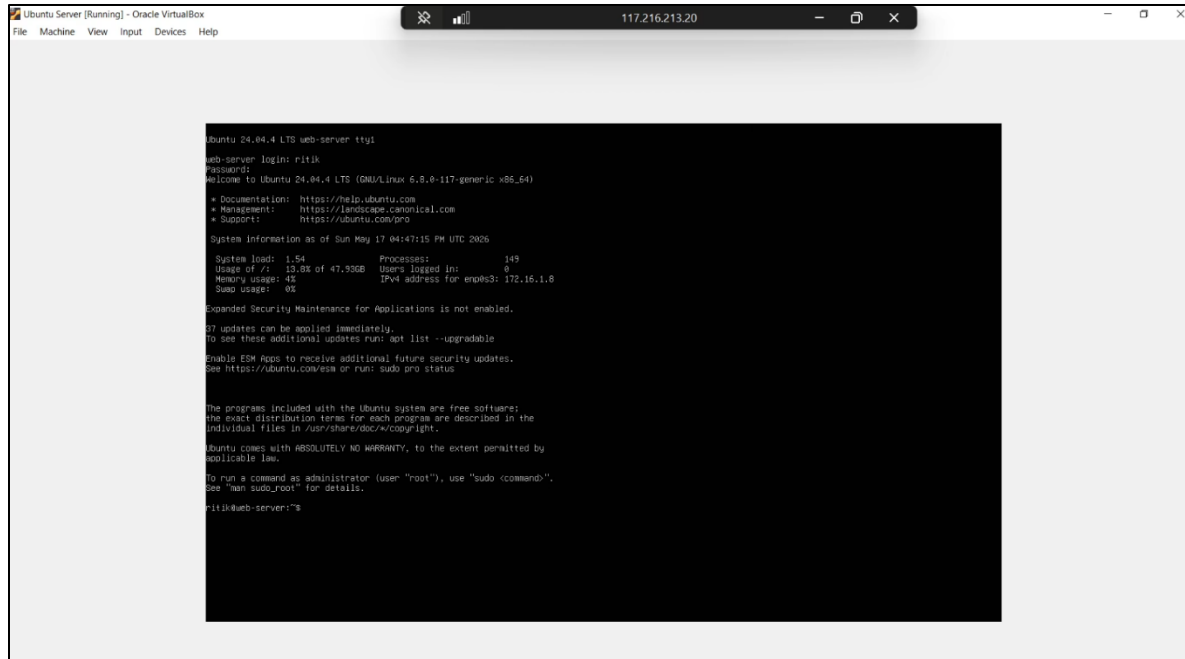


3.3 Ubuntu Server Installation

Ubuntu Server was installed using the normal installation process. During the installation, OpenSSH server was installed and a user account was created and the server hostname was configured as:

web-server

After installation, the server was restarted and the Ubuntu login screen was displayed. This confirmed that the Ubuntu Server installation was completed successfully.



3.4 Updating Ubuntu Server

After logging into Ubuntu Server, the package list was updated and installed packages were upgraded.

Commands Used:

```
sudo apt update
```

```
sudo apt upgrade -y
```

Explanation:

- `sudo apt update` updates the package list from Ubuntu repositories
- `sudo apt upgrade -y` upgrades installed packages automatically

```
ritik@web-server:~$ sudo apt update
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
37 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

</Hacker4Help>

```
ritik@web-server:~$ sudo apt upgrade -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following packages will be upgraded:
  apparmor cloud-init coreutils distro-info-data fwupd iproute2 libapparmor1 libfwupd2 libnetplan
  libsystemd-shared libsystemd0 libudev1 linux-base lshw netplan-generator netplan.io nftables op
  rsyslog snapd software-properties-common sosreport systemd systemd-dev systemd-hwe-hwdb system
  ubuntu-drivers-common ubuntu-pro-client ubuntu-pro-client-l10n udev
37 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
Need to get 55.7 MB of archives.
After this operation, 1,588 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 coreutils amd64 9.4-3ubuntu6.2 [1
Get:2 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libnss-systemd amd64 255.4-1ubun
Get:3 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 systemd-dev all 255.4-1ubuntu8.15
Get:4 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 systemd-timesyncd amd64 255.4-1ub
Get:5 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 systemd-resolved amd64 255.4-1ubu
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libsystemd-shared amd64 255.4-1ub
Get:7 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libsystemd0 amd64 255.4-1ubuntu8
```

3.5 Installing Network Tools

The net-tools package was installed to use basic network commands such as ifconfig.

Command Used:

```
sudo apt install net-tools
```

Explanation:

The net-tools package helps in checking network configuration and identifying the server IP address.

```
ritik@web-server:~$ sudo apt install net-tools
[sudo] password for ritik:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  net-tools
0 upgraded, 1 newly installed, 0 to remove and 37 not upgraded.
Need to get 204 kB of archives.
After this operation, 811 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 net-tools amd64 2.10-0.1ubuntu4.4 [204 kB]
Fetched 204 kB in 0s (882 kB/s)
Selecting previously unselected package net-tools.
(Reading database ... 88203 files and directories currently installed.)
Preparing to unpack .../net-tools_2.10-0.1ubuntu4.4_amd64.deb ...
Unpacking net-tools (2.10-0.1ubuntu4.4) ...
Setting up net-tools (2.10-0.1ubuntu4.4) ...
Processing triggers for man-db (2.12.0-4build2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ritik@web-server:~$ _
```

3.6 Verifying Internet Connectivity

Internet connectivity was tested from the Ubuntu Server using a ping command.

Command Used:

```
ping 8.8.8.8
```

Explanation:

This command checks whether the Ubuntu Server can communicate with an external IP address. A successful reply confirms that the server has internet connectivity.

```
ritik@web-server:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=255 time=25.9 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=255 time=25.9 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=255 time=25.9 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=255 time=26.1 ms
^C
--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3062ms
rtt min/avg/max/mdev = 25.865/25.933/26.086/0.089 ms
ritik@web-server:~$ _
```

3.7 Identifying Ubuntu Server IP Address

The IP address of the Ubuntu Server was checked using the following command:

Ritik Patel

SOC - Threat Research intern at </Hacker4Help>

ifconfig

```
ritik@web-server:~$ ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.16.1.8 netmask 255.255.255.0 broadcast 172.16.1.255
    inet6 fe80::a00:27ff:fe31:c2db prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:31:c2:db txqueuelen 1000 (Ethernet)
    RX packets 235 bytes 244295 (244.2 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 227 bytes 18468 (18.4 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 106 bytes 8843 (8.8 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 106 bytes 8843 (8.8 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

ritik@web-server:~$ _
```

3.8 Verifying Connectivity from Kali Linux

After identifying the Ubuntu Server IP address, connectivity was tested from Kali Linux.

Command Used from Kali Linux:

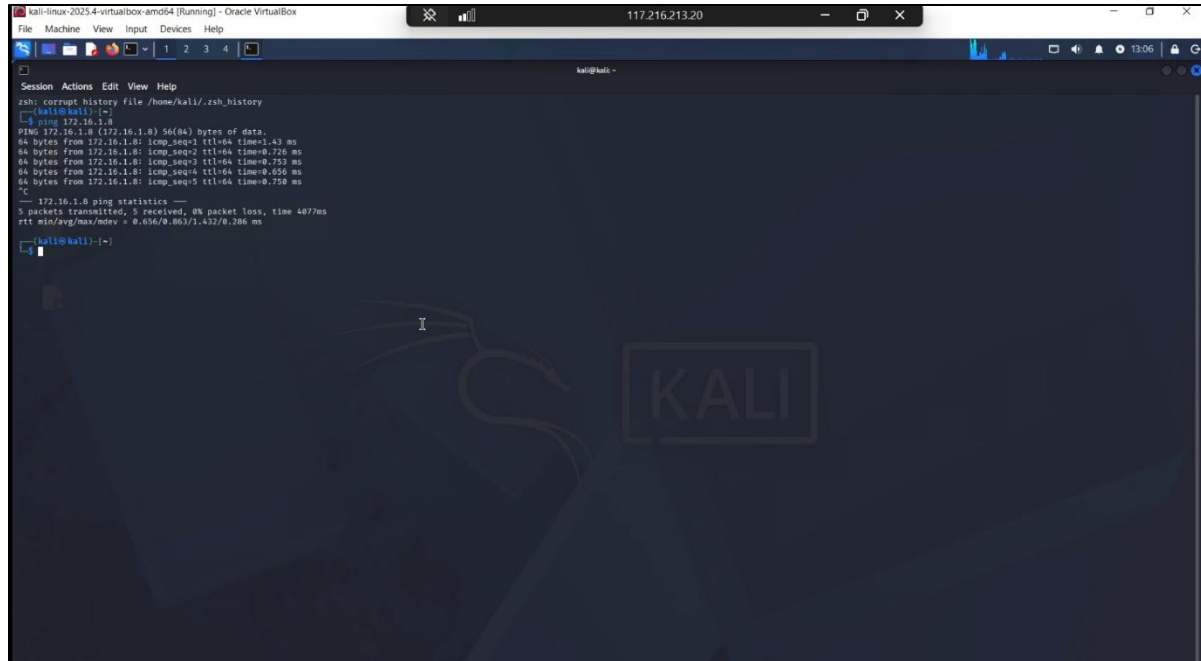
```
ping 172.16.1.8
```

Explanation:

This command verifies that Kali Linux can communicate with the Ubuntu Server inside the SOC lab network.

Observation:

Kali Linux successfully pinged the Ubuntu Server. This confirmed that both machines were connected to the same SOC lab network and could communicate with each other.



The screenshot shows a terminal window titled 'kali-linux-2023.4-virtualbox-amd64 [Running] - Oracle VM VirtualBox'. The terminal output includes the following commands and results:

```
zsh: corrupt history file /home/kali/.zsh_history
kali@kali:~$ ping 172.16.1.8
PING 172.16.1.8 (172.16.1.8) 56(64) bytes of data:
64 bytes from 172.16.1.8: icmp_seq=1 ttl=64 time=1.43 ms
64 bytes from 172.16.1.8: icmp_seq=2 ttl=64 time=0.726 ms
64 bytes from 172.16.1.8: icmp_seq=3 ttl=64 time=0.733 ms
64 bytes from 172.16.1.8: icmp_seq=4 ttl=64 time=0.656 ms
64 bytes from 172.16.1.8: icmp_seq=5 ttl=64 time=0.750 ms
^C
--- 172.16.1.8 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 487ms
rtt min/avg/max/mdev = 0.636/0.863/1.432/0.286 ms
kali@kali:~$
```

3.9 Outcome

After completing the Ubuntu Server installation and network verification, the new web server was successfully integrated into the SOC lab environment.

4. Docker Installation on Ubuntu Server

4.1 Overview

After successfully installing and configuring the Ubuntu Server, Docker was installed on it. Docker was required because DVWA was deployed as a containerized web application instead of being installed directly on the Ubuntu host.

Using Docker makes the lab more flexible because multiple web applications can be hosted on the same Ubuntu Server in isolated containers without affecting each other.

4.2 Installing Required Packages

Before installing Docker, required packages were installed on the Ubuntu Server.

Commands Used:

```
sudo apt-get update
```

```
sudo apt-get install -y ca-certificates curl
```

Explanation:

- ca-certificates allows the system to verify secure HTTPS connections
- curl is used to download files from external repositories
- These packages are required before adding the official Docker repository

```
ritik@web-server:~$ sudo apt-get update
[sudo] password for ritik:
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
ritik@web-server:~$ _
```

```
ritik@web-server:~$ sudo apt-get install ca-certificates curl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ca-certificates is already the newest version (20240203).
ca-certificates set to manually installed.
curl is already the newest version (8.5.0-2ubuntu10.9).
curl set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
ritik@web-server:~$ _
```

4.3 Adding Docker GPG Key

The Docker GPG key was added to verify packages downloaded from the Docker repository.

Commands Used:

```
sudo install -m 0755 -d /etc/apt/keyrings
```

```
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
```

```
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

Explanation:

These commands create a secure keyring directory, download Docker's official GPG key, and set the correct permissions for the key file.

```
ritik@web-server:~$ sudo install -m 0755 -d /etc/apt/keyrings
ritik@web-server:~$ sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
ritik@web-server:~$ sudo chmod a+r /etc/apt/keyrings/docker.asc
```

4.4 Adding Docker Repository

After adding the GPG key, the Docker repository was added to the Ubuntu Server.

Command Used:

```
echo "deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu noble stable" | sudo tee
/etc/apt/sources.list.d/docker.list
```

After adding the repository, the package list was updated again.

</Hacker4Help>

```
sudo apt-get update
```

Explanation:

This command adds the Docker repository for Ubuntu 24.04, which uses the codename noble. This allows Docker packages to be installed directly through the apt package manager.

```
ritik@web-server:~$ echo "deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu noble stable" | sudo tee /etc/apt/sources.list.d/docker.list
deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu noble stable
ritik@web-server:~$ _
```

```
ritik@web-server:~$ sudo apt-get update
Get:1 https://download.docker.com/linux/ubuntu noble InRelease [48.5 kB]
Hit:2 http://security.ubuntu.com/ubuntu noble-security InRelease
Get:3 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [54.1 kB]
Hit:4 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:5 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:6 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Fetched 103 kB in 1s (97.6 kB/s)
Reading package lists... Done
ritik@web-server:~$ _
```

4.5 Installing Docker Engine and Docker Compose

After the Docker repository was configured, Docker Engine and Docker Compose plugin were installed.

Command Used:

```
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
```

Explanation:

- docker-ce installs Docker Community Edition
- docker-ce-cli installs Docker command-line tools
- containerd.io provides the container runtime
- docker-buildx-plugin supports advanced Docker builds
- docker-compose-plugin allows running multi-container applications using Docker Compose

Ritik Patel

SOC - Threat Research intern at </Hacker4Help>

</Hacker4Help>

```
ritik@web-server:~$ sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  docker-ce-rootless-extras pigz
Suggested packages:
  cgroupfs-mount | cgroup-lite docker-model-plugin
The following NEW packages will be installed:
  containerd.io docker-buildx-plugin docker-ce docker-ce-cli docker-ce-rootless-extras docker-compose-plugin pigz
0 upgraded, 7 newly installed, 0 to remove and 0 not upgraded.
Need to get 98.9 MB of archives.
After this operation, 378 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/universe amd64 pigz amd64 2.8-1 [65.6 kB]
Get:2 https://download.docker.com/linux/ubuntu noble/stable amd64 containerd.io amd64 2.2.3-1~ubuntu.24.04~noble [23.6 MB]
Get:3 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce-cli amd64 5:29.5.0-1~ubuntu.24.04~noble [17.0 MB]
Get:4 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce amd64 5:29.5.0-1~ubuntu.24.04~noble [22.9 MB]
Get:5 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-buildx-plugin amd64 0.34.0-1~ubuntu.24.04~noble [17.2 MB]
Get:6 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce-rootless-extras amd64 5:29.5.0-1~ubuntu.24.04~noble [10
Get:7 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-compose-plugin amd64 5.1.3-1~ubuntu.24.04~noble [8,023 kB]
```

4.6 Starting and Enabling Docker Service

After installation, the Docker service was started and enabled to run automatically when the server starts.

Commands Used:

```
sudo systemctl start docker
```

```
sudo systemctl enable docker
```

The Docker service status was checked using:

```
sudo systemctl status docker
```

Explanation:

- start docker starts the Docker service
- enable docker ensures Docker starts automatically after reboot
- status docker verifies whether Docker is running

```
ritik@web-server:~$ sudo systemctl start docker
ritik@web-server:~$ sudo systemctl enable docker
Synchronizing state of docker.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable docker
ritik@web-server:~$
```

4.7 Verifying Docker Installation

After installation, Docker was verified using version and test commands.

Commands Used:

```
sudo docker --version
```

Ritik Patel

SOC - Threat Research intern at </Hacker4Help>

</Hacker4Help>

```
sudo docker compose version
```

```
sudo docker run hello-world
```

Explanation:

- docker --version confirms Docker Engine installation
- docker compose version confirms Docker Compose plugin installation
- docker run hello-world runs a test container to confirm Docker is working properly

```
ritik@web-server:~$ sudo docker --version
Docker version 29.5.0, build 98f1464
ritik@web-server:~$ sudo docker compose version
Docker Compose version v5.1.3
```

```
ritik@web-server:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:0e760dfdbc48ba8041e7c6db999bb40bfca508b4be580ac75d32c4e29d202ce1
Status: Downloaded newer image for hello-world:latest
```

4.8 Outcome

Docker was successfully installed and verified on the Ubuntu Server.

Ritik Patel

SOC - Threat Research intern at </Hacker4Help>

5. DVWA Deployment Using Docker

5.1 Overview

After Docker was successfully installed on the Ubuntu Server, DVWA was deployed as a containerized vulnerable web application. DVWA was deployed as the target web application that will be used in the next phase for controlled web attack simulation such as brute force and SQL injection.

Docker was used because it allows DVWA and its database to run in isolated containers without directly affecting the Ubuntu host system or other future web applications.

5.2 Downloading DVWA from GitHub

DVWA was downloaded from its official GitHub repository.

Commands Used:

```
cd ~
```

```
git clone https://github.com/digininja/DVWA.git
```

```
cd DVWA
```

Explanation:

- `cd ~` moves to the home directory
- `git clone` downloads the DVWA project files
- `cd DVWA` enters the DVWA project directory

```
ritik@web-server:~$ cd ~
ritik@web-server:~$ pwd
/home/ritik
ritik@web-server:~$ git clone https://github.com/digininja/DVWA.git
Cloning into 'DVWA'...
remote: Enumerating objects: 5720, done.
remote: Counting objects: 100% (44/44), done.
remote: Compressing objects: 100% (25/25), done.
remote: Total 5720 (delta 30), reused 19 (delta 19), pack-reused 5676 (from 3)
Receiving objects: 100% (5720/5720), 2.73 MiB | 6.00 MiB/s, done.
Resolving deltas: 100% (2849/2849), done.
ritik@web-server:~$ cd DVWA/
```

Ritik Patel

SOC - Threat Research intern at </Hacker4Help>

5.3 Starting DVWA Containers

After downloading DVWA, Docker Compose was used to start the DVWA application.

Command Used:

```
sudo docker compose up -d
```

Explanation:

- docker compose reads the DVWA Docker Compose configuration file
- up creates and starts the required containers
- -d runs the containers in the background

This command starts the DVWA web application container and its database container.

```
ritik@web-server:~/DVWA$ sudo docker compose up -d
[+] up 36/36
* Image docker.io/library/mariadb:10 Pulled
* Image ghcr.io/digininja/dvwa:latest Pulled
* Network dvwa_dvwa Created
* Volume dvwa_dvwa Created
* Container dvwa-db-1 Started
* Container dvwa-dvwa-1 Started
ritik@web-server:~/DVWA$ if
```

5.4 Verifying Running Containers

After starting DVWA, the running containers were checked.

Command Used:

```
sudo docker ps
```

Explanation:

This command displays currently running Docker containers.

Observation:

Two main containers were running:

- DVWA web application container
- DVWA database container

</Hacker4Help>

This confirmed that DVWA and its database were running properly.

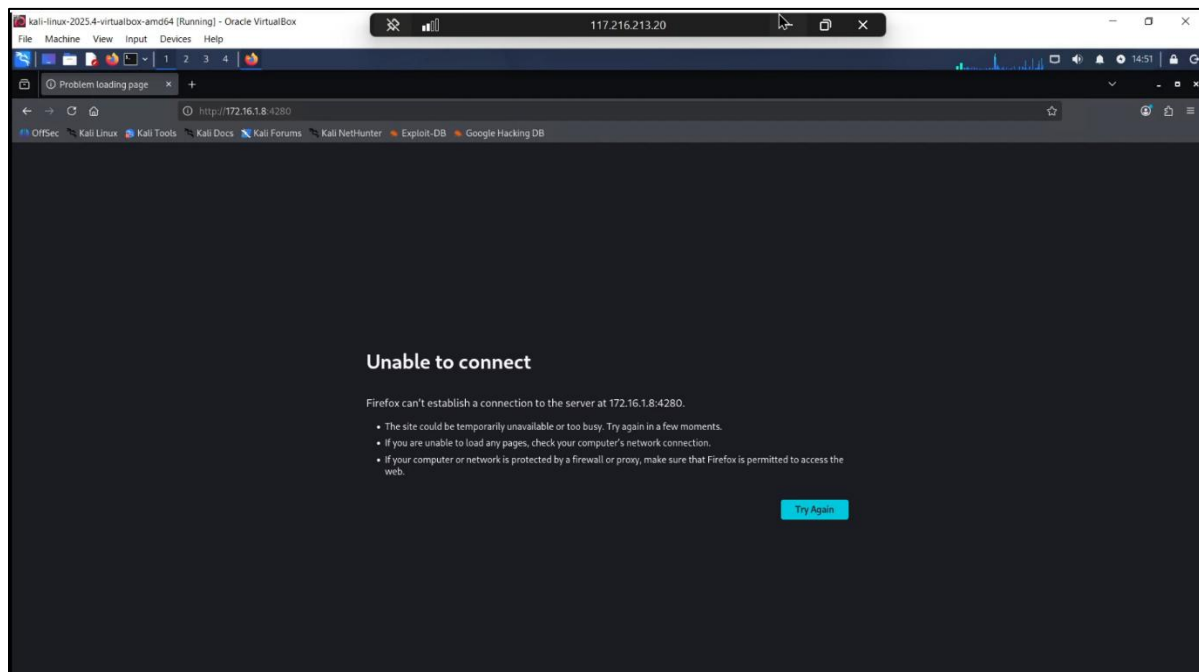
```
ritik@web-server:~/DVWA$ sudo docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                    NAMES
20965d7d711b   ghcr.io/digininja/dvwa:latest      "docker-php-entrypool..." 4 minutes ago  Up 4 minutes  127.0.0.1:4280->80/tcp    dvwa-dvwa-1
f7979be9848e   mariadb:10                         "docker-entrypoint.s..." 4 minutes ago  Up 4 minutes  3306/tcp                 dvwa-db-1
ritik@web-server:~/DVWA$
```

5.5 Fixing DVWA Network Access

Initially, DVWA was only listening on localhost:

127.0.0.1:4280

Because of this, Kali Linux could not access DVWA from another machine in the SOC lab network.



To fix this, the Docker Compose port binding was updated so DVWA could listen on all network interfaces.

The Docker Compose file was opened:

nano compose.yml

```
ritik@web-server:~/DVWA$ nano compose.yml_
```

The port mapping was changed from:

Ritik Patel

SOC - Threat Research intern at </Hacker4Help>

</Hacker4Help>

127.0.0.1:4280:80

```
- dvwa
ports:
- 127.0.0.1:4280:80
restart: unless-stopped
```

to:

4280:80

```
networks:
- dvwa
ports:
- 4280:80
restart: unless-stopped
```

After saving the file, DVWA containers were restarted.

Commands Used:

```
sudo docker compose down
```

```
sudo docker compose up -d
```

Explanation:

- docker compose down stops and removes the existing containers
- docker compose up -d recreates and starts them again with the updated port mapping

Observation:

After updating the port binding, DVWA became accessible from Kali Linux using the Ubuntu Server IP address.

```
ritik@web-server:~/DVWA$ sudo docker compose down
[+] down 3/3
✦ Container dvwa-dvwa-1 Removed
✦ Container dvwa-db-1 Removed
✦ Network dvwa_dvwa Removed
ritik@web-server:~/DVWA$ sudo docker compose up -d
[+] up 4/4
✦ Image ghcr.io/digininja/dvwa:latest Pulled
✦ Network dvwa_dvwa Created
✦ Container dvwa-db-1 Started
✦ Container dvwa-dvwa-1 Started
ritik@web-server:~/DVWA$ _
```

Ritik Patel

SOC - Threat Research intern at </Hacker4Help>

</Hacker4Help>

5.6 Accessing DVWA from Kali Linux

After fixing the port binding, DVWA was accessed from Kali Linux using a web browser.

URL Used:

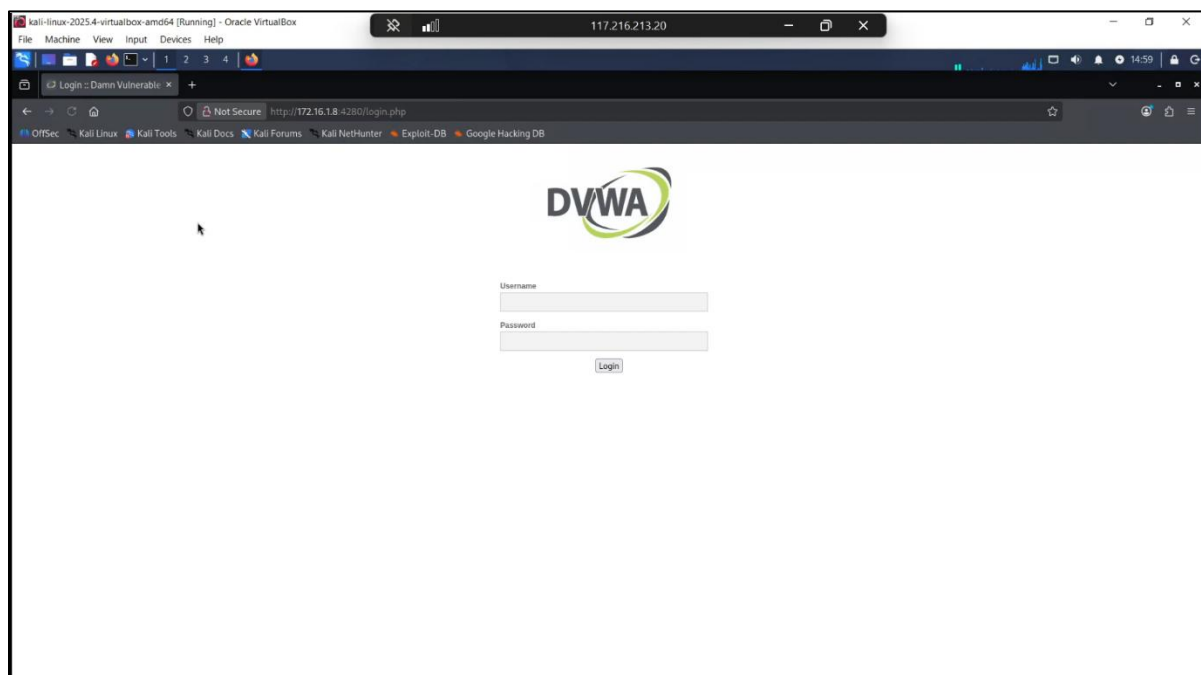
`http://172.16.1.8:4280`

Explanation:

- 172.16.1.8 is the Ubuntu web server IP address
- 4280 is the host port mapped to the DVWA web container

Observation:

DVWA opened successfully in the Kali Linux browser, confirming that the web application was reachable from the attacker machine.



5.7 DVWA Login and Database Setup

After opening DVWA, the default credentials were used to log in.

Default Credentials:

Username: admin

Ritik Patel

SOC - Threat Research intern at </Hacker4Help>

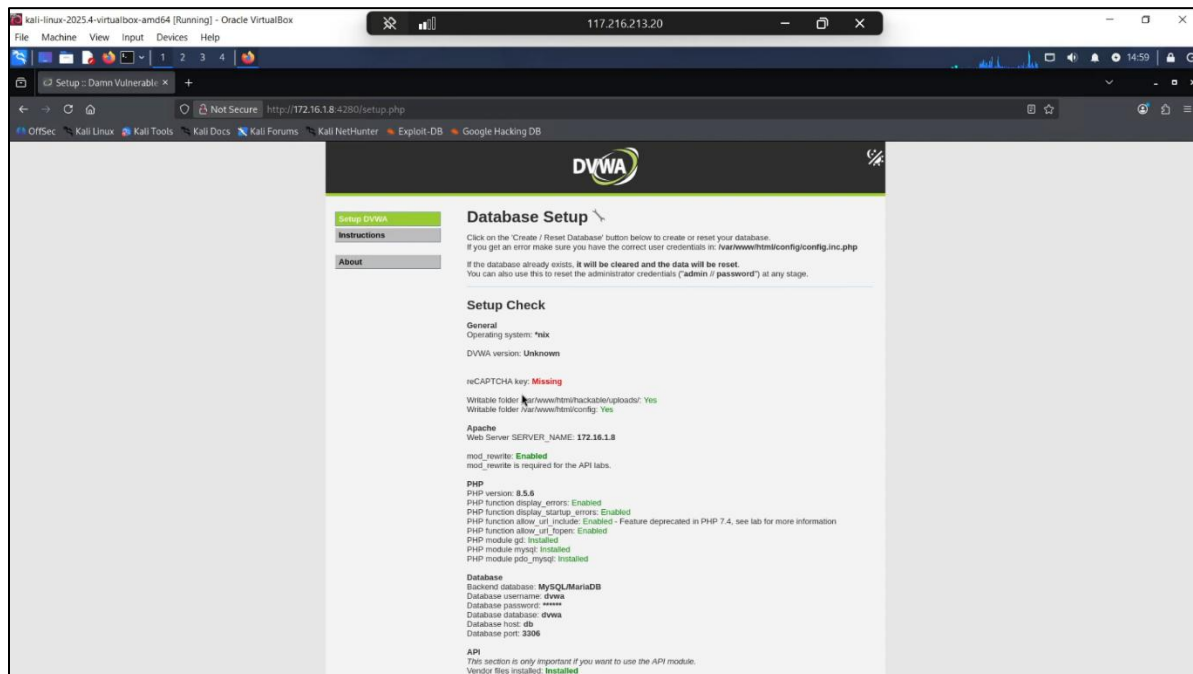
</Hacker4Help>

Password: password

After login, the database was initialized from the DVWA setup page.

Steps Performed:

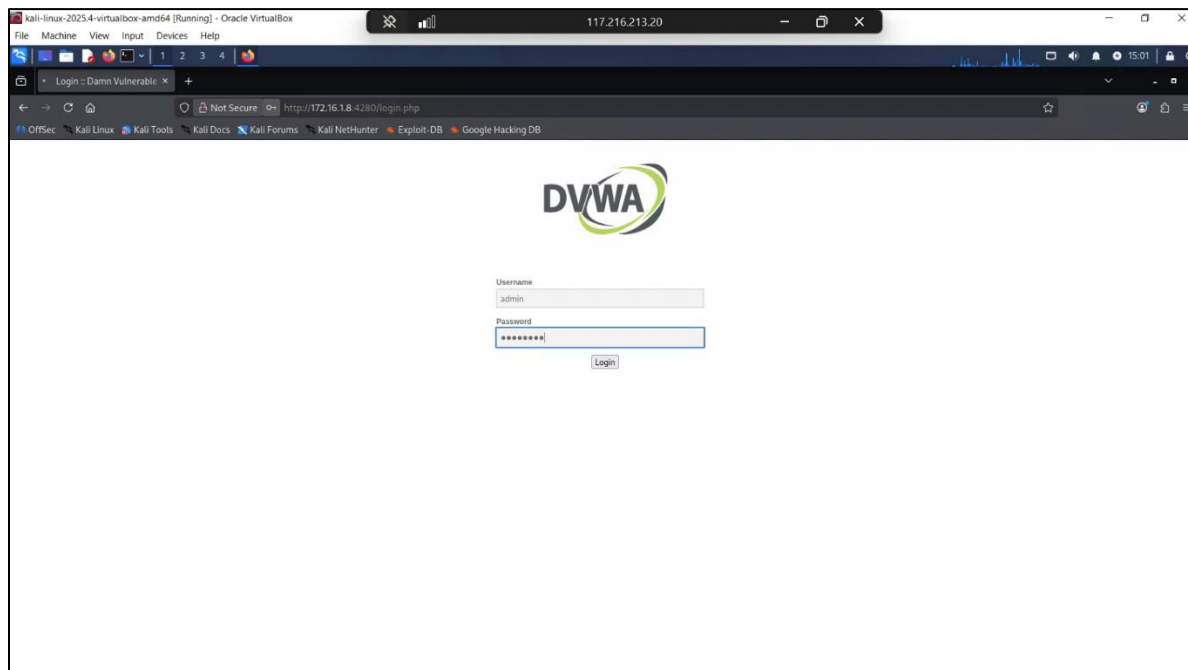
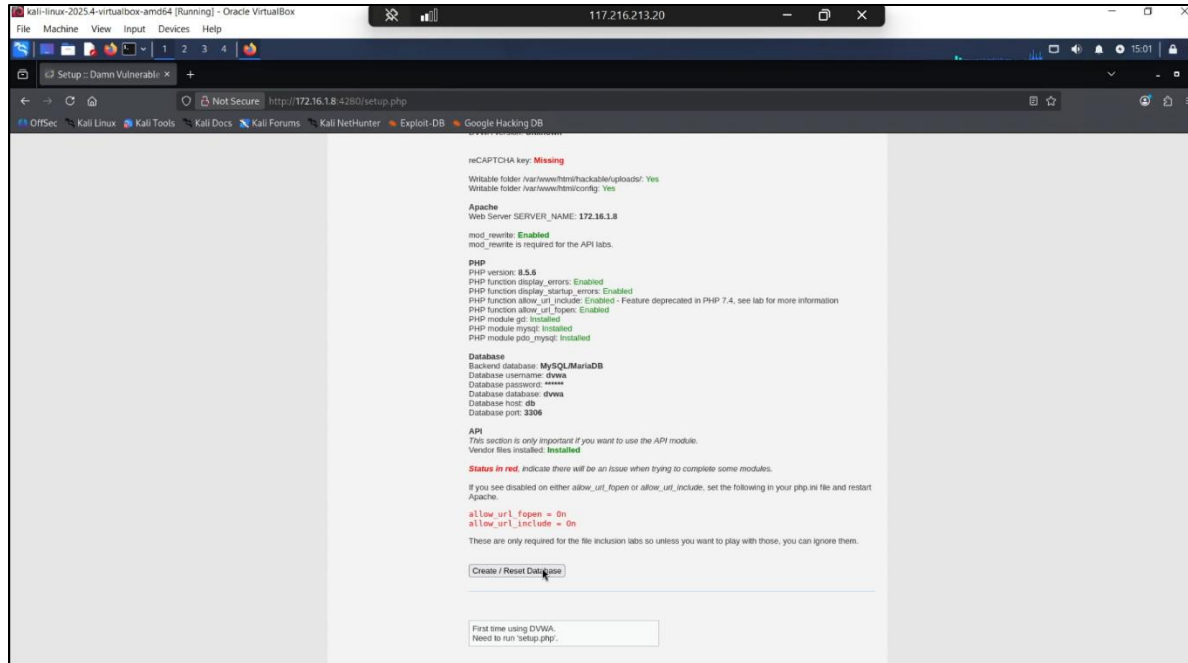
1. Open DVWA in Kali browser
2. Login using default credentials
3. Go to Setup / Reset DB
4. Click Create / Reset Database
5. Login again if required



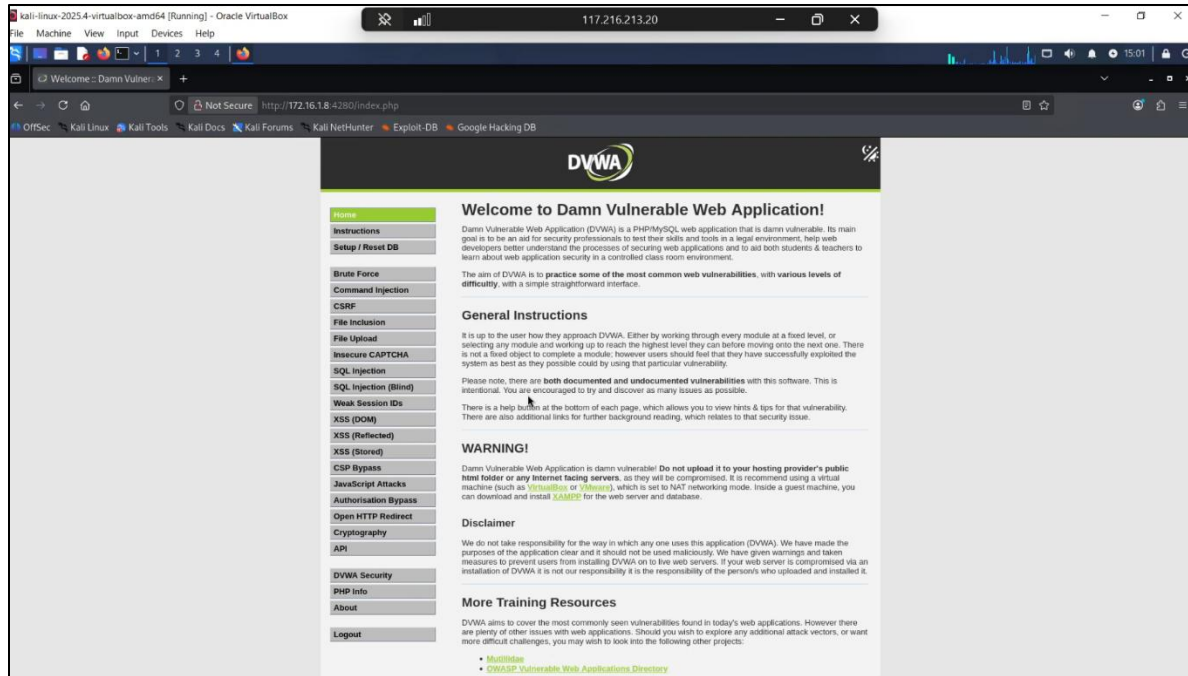
Ritik Patel

SOC - Threat Research intern at </Hacker4Help>

</Hacker4Help>



Ritik Patel
SOC - Threat Research intern at </Hacker4Help>



5.8 Setting DVWA Security Level

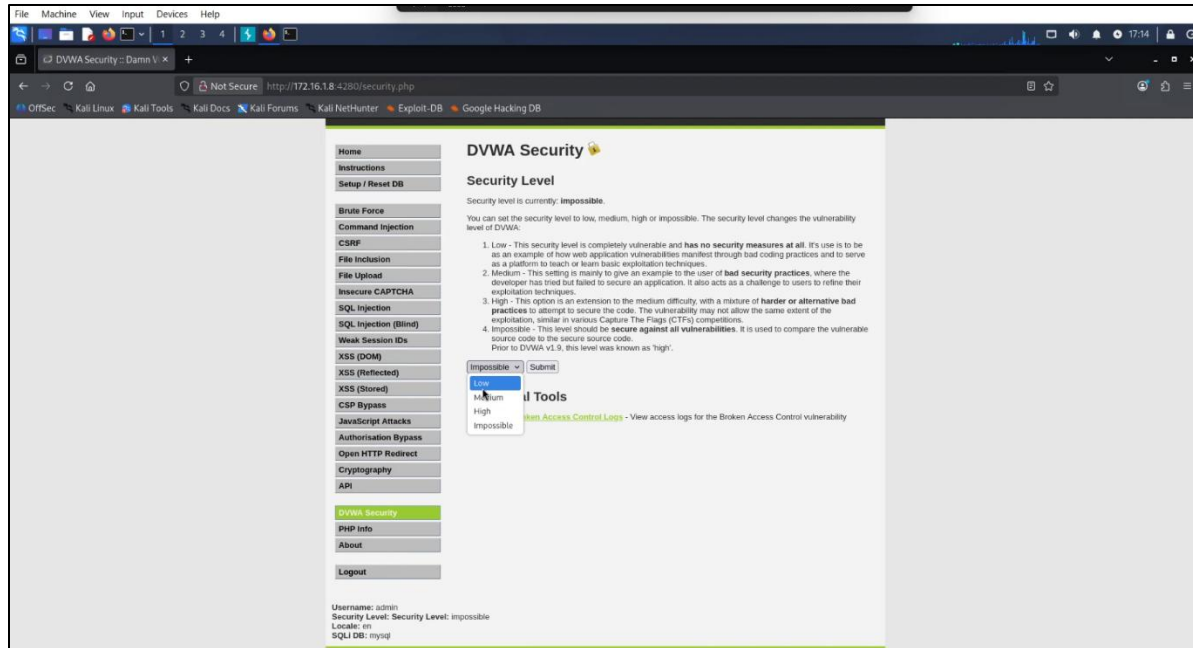
Before performing attacks, the DVWA security level was changed to Low.

Steps Performed:

1. Open DVWA
2. Go to DVWA Security
3. Select Low
4. Click Submit

Explanation:

The low security level was selected to prepare DVWA for future controlled attack simulation and detection testing.



5.9 Outcome

DVWA was successfully deployed on the Ubuntu Server using Docker and accessed from Kali Linux.

6. Conclusion

6.1 Summary

In this part of the project, a dedicated Ubuntu web server was successfully integrated into the existing SOC lab environment. The server was installed in VirtualBox, connected to the same NAT network as the other lab machines, and verified for internet and internal network connectivity.

Docker was installed on the Ubuntu Server to support containerized application deployment. After Docker installation, DVWA was deployed using Docker Compose. DVWA was configured to run as a vulnerable web application with a separate database container, creating a realistic web application environment for future attack simulation.

The DVWA port binding was also adjusted so that the application could be accessed from Kali Linux using the Ubuntu Server IP address. This confirmed that the web application was reachable from another machine inside the SOC lab network.

6.2 Key Achievements

- ✓ Successfully installed Ubuntu Server in VirtualBox
- ✓ Connected Ubuntu Server to the existing SOC lab network
- ✓ Verified internet connectivity from Ubuntu Server
- ✓ Verified communication between Kali Linux and Ubuntu Server
- ✓ Installed Docker and Docker Compose successfully
- ✓ Deployed DVWA using Docker Compose
- ✓ Verified DVWA web and database containers were running
- ✓ Fixed DVWA port binding for network access
- ✓ Accessed DVWA successfully from Kali Linux

</Hacker4Help>

6.3 Practical Learning Outcomes

This setup provided hands-on experience with:

- Ubuntu Server installation and configuration
- VirtualBox SOC lab networking
- Docker installation on Linux
- Containerized web application deployment
- DVWA setup and database initialization
- Troubleshooting container port binding issues
- Preparing a web application target for future SOC monitoring

6.4 Next Phase

The next report will focus on the detection side of the project. In that phase, the Wazuh agent will be installed on the Ubuntu web server, Docker container log collection will be configured, custom Wazuh rules will be created, and controlled attacks such as brute force and SQL injection will be performed against DVWA.